Prezime i ime učenika : *Mihajlo Stoiljković*

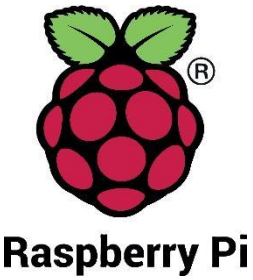
Odeljenje i grupa : *IV-4 grupa 3.*

R. broj i naziv vežbe : *Vežba 9. Rad sa digitalnim ulazima i izlazima*

IZVEŠTAJ:

Mikroračunar Raspberry Pi

Mikroračunar je mali digitalni računar koji se sastoji iz osnovnih računarskih komponenti (procesor, grafička kartica, unutrašnja memorija, spoljašnja memorija, IO jedinice..). Za razliku od mikrokontrolera (recimo Arduino), na mikroračunaru se može instalirati operativni sistem koji podržava u zavisnosti od arhitekture procesora (**8, 16, 32, 64 bita magistala**), danas najčešće noviji mikroračunari imaju 64bita arhitekturu. Pored toga, sastoji se iz grafičke kartice (integrisane) koja omogućava renderovanje animacija, **3D modela** i drugih video signala. Jedan od najpoznatijih mikroračunara jeste Raspberry Pi, koji služi za programiranje, industriju, robotiku, elektroniku i slično. Baziran je na Debian linux distribuciji (**Raspberry Pi OS**). Ranije je to bio Rasbian. U odnosu na mikrokontroler, poprilično ima veće mogućnosti i dosta je snažniji, ali je takođe i skuplji. Osnovne karakteristike Raspberry Pi 400 modela je prikazan ispod.



Procesor - Broadcom BCM2711 quad-core Cortex-A72 (ARM v8) 64-bit SoC @ **1.8 GHz** — jači od standardnog Pi 4 zbog boljeg hlađenja u kućištu tastature.

Memorija (RAM) - 4 GB LPDDR4-3200 integrisane memorije (nije proširiva).

Povezivost - Wi-Fi: Dual-band 802.11b/g/n/ac (2.4 GHz i 5 GHz), **Bluetooth:** Bluetooth 5.0 + BLE, **Ethernet: Gigabit Ethernet** (žična mreža)

Portovi

- **2 × USB 3.0**
- **1 × USB 2.0**
- **USB-C** za napajanje (5 V DC).
- **RJ45 Gigabit Ethernet** port.

Video i zvuk

- **2 × micro HDMI** izlaza, podržava **do 4Kp60** rezoluciju (npr. dual-monitor setup).
- GPU sa hardverskim dekodiranjem/enkodiranjem:
  - H.265 (4Kp60 dekodiranje)
  - H.264 (1080p60 dekodiranje, 1080p30 enkodiranje)

- **OpenGL ES 3.x** grafika.

Skladištenje

- **microSD kartica** slot za OS i podatke.

GPIO - Horizontalni 40-pinskih GPIO header za dodatne module/projekte (HAT-ove)

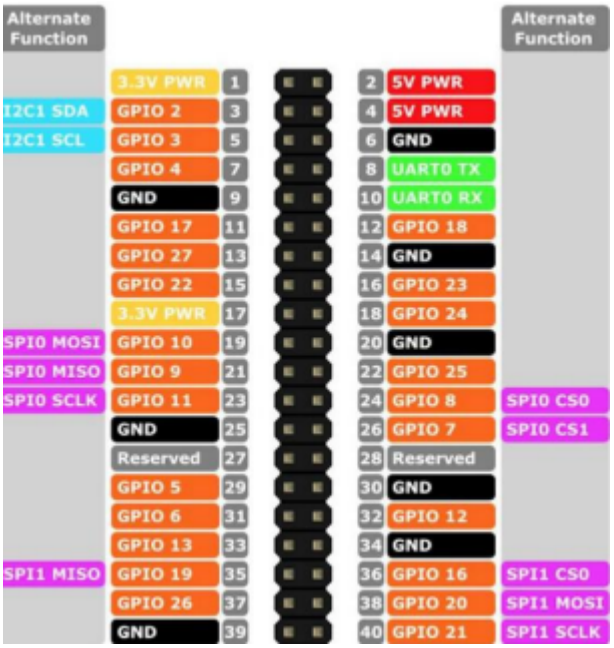
Tastatura - Ugrađena kompaktna tastatura (78- ili 79-tastera) — Pi 400 je uklopljen u oblik tastature.

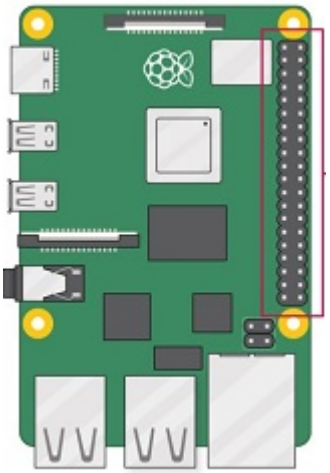
Radni uslovi & dimenzije

- **Radna temperatura:** 0 °C do +50 °C.
- **Dimenzije:** oko **286 × 122 × 23 mm**. ž

GPIO pinovi i karakteristike

GPIO (general purpose input output) su univerzalni pinovi koji mogu biti ulazni ili izlazni u zavisnosti od naše konfiguracije koje definišemo unutar programa. Raspberry Pi mikroračunar se sastoji iz 40 GPIO pinova koji uključuju i naponske pinove (GND, VCC) kao i drugi pinovi koji služe za neke od serijskih komunikacija. Pinovi 26 I 27 su rezervisani za I2C magistralu.

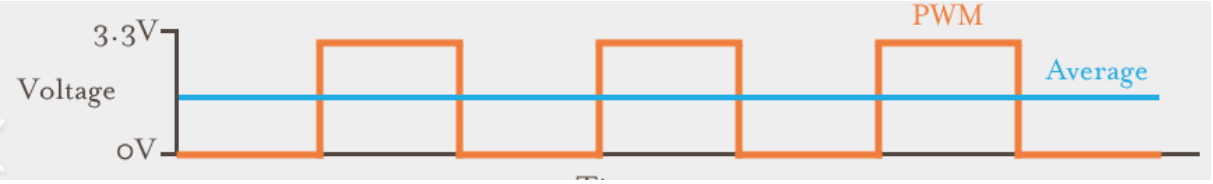




Na samoj RPi ploči nemamo tačne oznake koji pin za šta služi, pa može biti frustrirajuće prilikom povezivanja. Kao rešenje postoji GPIO header koji se sastoji iz 40 pina gde postoje jasne oznake za svaki pin za šta on služi. Što se tiče numeracije pinova, pinovi kod RPi mogu biti numerisani prema procesoru (**BCM** - Broadcom) ili prema redosledu kako idu na ploči (1, 2, 3, 4 -> **BOARD**). RPi ne poseduje analogne ulazne GPIO pinove, pa kako bi koristili analogni pin odnosno kako bi imali mogućnost korišćenja potrebno je digitalizovati analogni napon upotrebnom analogno digitalnog konvertora (A/D). Primer za to je čip **MCP3008**. Drugi način gde možemo “simulirati” analogni pin

preko digitalnog jeste korišćenjem **PWM** signala (Pulse Width modulation). PWM signali su zapravo signali koji imaju mogućnost menjana dužine trajanja logičke nule i jedinice u jednom period signala, što čini dati pin da se ponaša kao analogni pin. Važno je napomenuti da nemaju svi GPIO pinovi tu mogućnost, već pojedini. Taj odnos se naziva kao radni ciklus odnosno duty cycle.

- Primer:
  - 0% duty cycle → stalno 0 V
  - 50% duty cycle → „kao“ ~2.5 V (ako je 5 V logika)
  - 100% duty cycle → stalno 5 V



Pored ovih karakteristika, važno je reći da RPi radi sa GPIO pinovima na 3.3V. Za razliku od mikrokontrolera Arduino koji nativno podržava 5V, mikroračunar Raspberry Pi ne podržava 5V već 3.3V. Ukoliko se dovede 5V napon na nekom od GPIO pinova, najčešće dolazi do oštećenja procesora, pa samim ti i celog mikroračunara s obzirom da je procesor integrisan na ploči (SoC računar – ceo računar na jednoj ploči).

Kontrola GPIO pinova i programiranje

Kontrola GPIO pinova zahteva složeniji kod, ali postoji mogućnost primene određenih biblioteka/modula koji imaju gotove funkcije i ostale delove programa za rad koje su olakšica. Programski jezici koji se koriste za programiranje Raspberry Pi mikroračunara su C/C++, python i slično. Najčešće se koristi Python zbog svoje jednostavnosti i velikom broju biblioteka vezane za elektronske komponente i sam mikroračunar.





Na početku samog programa potrebno je da uvezemo python modul za GPIO pinove:

```
import RPi.GPIO as GPIO
```

Ovime importujemo modul RPi.GPIO, gde ćemo ga u programu zvati kao GPIO za šta nam i služi ključna reč as.

Nakon uvoženja modula, potrebno je reći python interpreteru (izvršava kod liniju po liniju) koju numeraciju pinova ćemo koristiti (BCM ili BOARD). Ova dva pojma smo napomenuli gore.

```
GPIO.setmode(GPIO.BCM)
```

Ili

```
GPIO.setmode(GPIO.BOARD)
```

Kao sledeći korak je potrebno da se podesi određeni pin kao ulazni ili izlazni. To se radi uz pomoć sledećih funkcija:

```
GPIO.setup(port_or_pin, GPIO.IN)
GPIO.setup(port_or_pin, GPIO.OUT)
```

U sledećim koracima sam program zavisi od toga kako želimo da naš sistem radi, koje će funkcije koristiti, kada se postavlja visok a kad nizak naponski nivo I slično. Za postavljanje izlaza na visok ili nizak naponski nivo se koristi funkcija:

```
GPIO.output(port_or_pin, GPIO.HIGH) # 1
GPIO.output(port_or_pin, GPIO.LOW) # 0
```

Umesto GPIO.HIGH ili GPIO.LOW može se staviti ili broj 1 koji označava logičku jedinicu ili 0 koji označava logičku nulu ili neka promenljiva koja menja stanje pina. Glavni program se uglavnom izvršava korišćenjem While petlje kao beskonačne petlje sa kombinacijom rukovanja izuzetaka (exception handling) kako se program nebi prekidao na neke greške i slično. Primer:

```
try:
    while True:
        GPIO.output(24, 1)
        sleep(0.5)
        GPIO.output(24, 0) #
        sleep(0.5)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Ovde vidimo da koristimo beskonačnu While petlju, vršimo postavljanje logične jedinice na pinu 24, zatim pravimo pauzu od pola sekunde korišćenjem sleep funkcije iz time modula k

Koji je potrebno uvesti

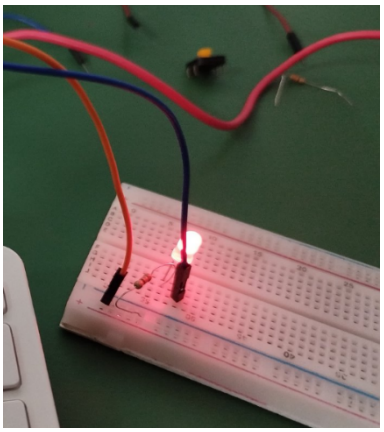
```
from time import sleep
```

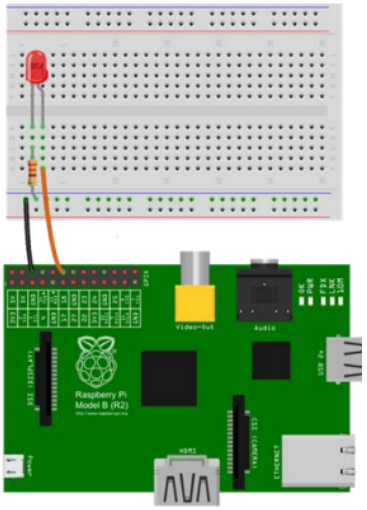
U ovom slučaju, importujemo samo funkciju sleep iz time modula, gde za bilo šta drugo je potrebno navoditi time modul pa zatim tačku pa ime funkcije (dot notation). Na kraju imamo `KeyboardInterrupt` izuzetak koji se postiže CTRL+C tasterima na tastaturi, i time se izvršava ***cleanup***() funkcija koja resetuje sve logične vrednosti GPIO pinova na početne odnosno default. Ukoliko se ta funkcija ne pozove, može doći do oštećenja procesora jer nisu vraćene logičke vrednosti na početne (0).

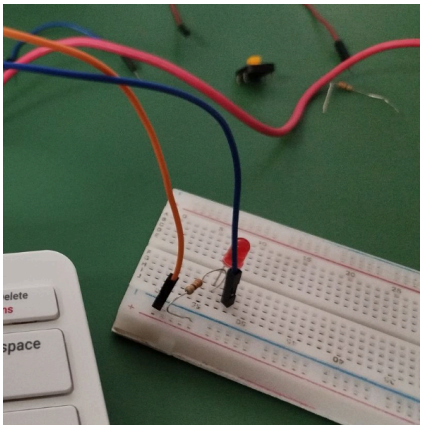
Prvi primer – uključivanje i isključivanje diode

Ovaj primer se realizuje korišćenjem Led diode, gde se tokom rada programa odnosno dok radi program dioda pali I gasi nakon određenog vremenskog intervala.

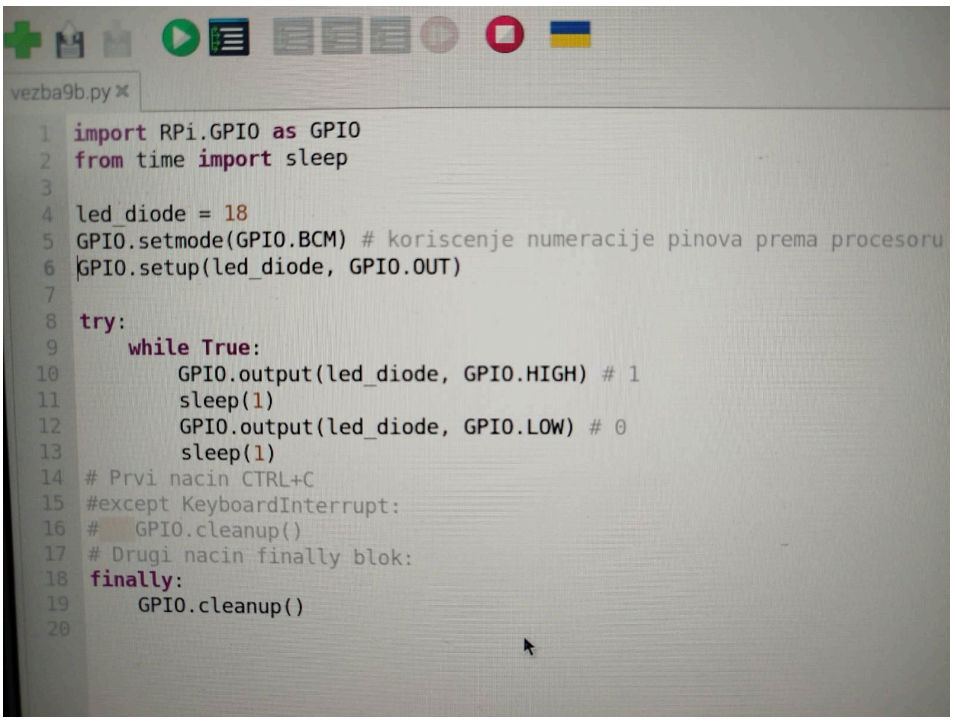
Povezivanje se vrši tako što LED anoda ide na GPIO pin 18 I LED katoda ide preko otpornika od 220 Ω na masu odnosno GND na RPi.







Program:

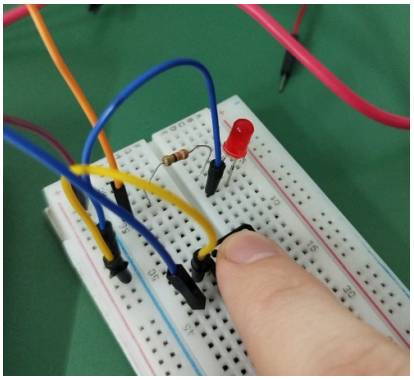


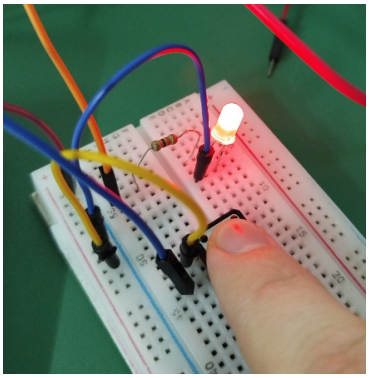
Opis programa:

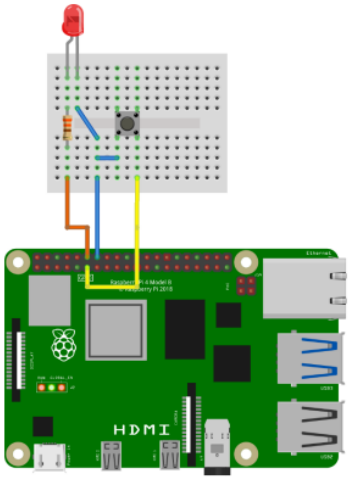
Na početku importujemo RPi.GPIO i time modul. Iz time modula importujemo samo sleep funkciju koja vrši odlaganje odnosno odziv (delay) programa dat u sekundama. Definisemo promenljivu led_diode na pinu 18 i definišemo tip numeracije pinova i postavljanje led_diode promenljive kao izlazne. U daljem programu se koristi rukovanje izuzecima, gde u Try bloku imamo glavni program koji se stalno izvršava pomoću beskonačne While petlje. Unutar petlje postavljamo pin led diode na logičnu jedinicu, zatim pravimo delay od 1 sekunde, postavljamo pin led diode na logičnu nulu, zatim opet delay 1 sekunda i to se stalno izvršava tokom rada programa. Program ukoliko koristimo `KeyboardInterrupt` izuzetak CTRL+C tasterima na tastaturi, gde se poziva funkcija cleanup() za postavljanje GPIO pinova na početno stanje (bezbedan izlaz).

Drugi primer – Led dioda pomoću tastera (polling)

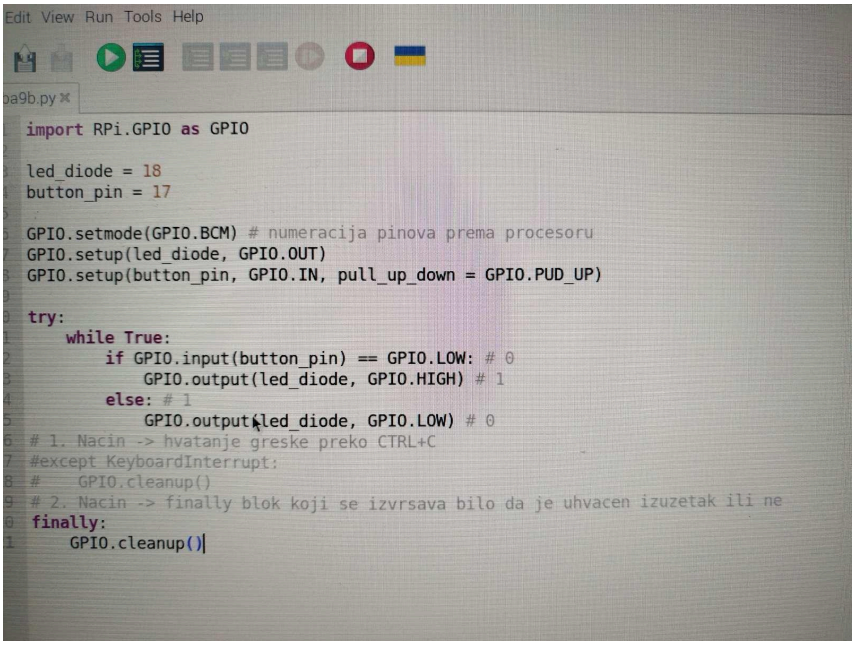
Ovaj primer se realizuje korišćenjem Led diode i tastera, gde tokom rada programa dioda kontrolise uz pomoć tastera koji radi u pull up režimu (Kad je pritisnut logička 0 suprotno 1). Povezivanje se vrši kao u prošlom primeru, s tim što dodajemo taster gde jedan pin ide na masu tj. GND a drugi ide na GPIO pin RPi-a.







Program:



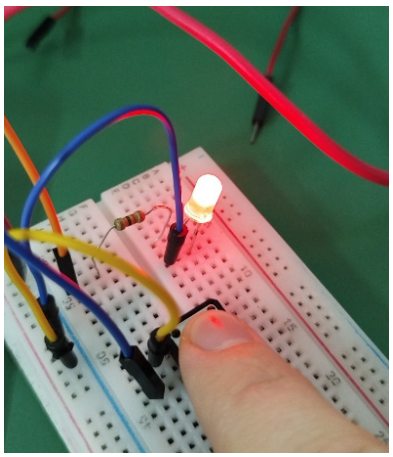
Opis programa:

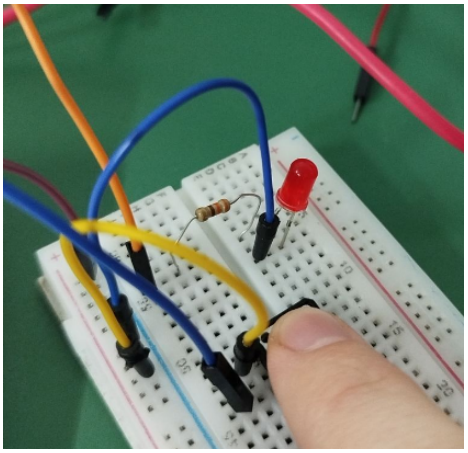
Na početku radimo isto kao i kod prošlog primera, s tim što ovde opterećujemo funkciju (overload) tako što kao dodatni argument setup funkcije stavljamo tip tastera (pull up / pull down). Nakon toga imamo obradu izuzetaka i beskonačnu petlju u kojoj preko uslova ispitujemo da li je button_pin jednak logičkoj nuli odnosno da li je pritisnut s obzirom da koristimo pull up konfiguraciju. Ukoliko jeste, led dioda se uključuje, u surptonom, onda se gasi. Ovo ispitivanje se vrši uz pomoć input funkcije. Na kraju kao i kod svakog RPi programa, pozivamo cleanup funkciju koja vrši resetovanje logičkih vrednosti GPIO pinova na početne.

Treći primer – hardverski prekid - uključivanje i isključivanje LED diode pritiskom na taster (interrupt)

Ovaj primer koristi iste elektronske komponente kao sa prošlog primera, s tim što je ovo efikasniji način rada jer se ne vrši stalna provera. Koristimo prekid gde kada se dugme pritisne, poziva se specijalna funkcija vezana za prekid i time izbegavamo stalno ispitivanje I pravimo program da bude efikasan.

Kada pritisnemo taster, on ostaje uključen sve dok opet se ne desi prekid odnosno ne pritisnemo taster gde se led dioda gasi I tako u krug. Šema za povezivanje je ista kao sa prošlog primera.





Program:

```python
import RPi.GPIO as GPIO
from time import sleep

led_diode = 18
button_pin = 17

GPIO.setmode(GPIO.BCM) # numeracija pinova prema procesoru
GPIO.setup(led_diode, GPIO.OUT)
GPIO.setup(button_pin, GPIO.IN, pull_up_down = GPIO.PUD_UP)

led_state = False

# Funkcija za ispis stanja dugmeta 0/1
def button_pressed(channel):
    # globalna bool promenljiva kako nebi kreirali lokalnu unutar funkcije
    global led_state
    # menjamo stanje led diode sa True na False i obratno
    led_state = not led_state
    # menjanje napona diode na logicku 1 ili 0 na osnovu led state
    GPIO.output(led_diode, led_state) # 0/1
    # stampanje led state u shell-u
    print(f"Taster pritisnut - LED stanje {led_state}")

GPIO.add_event_detect(
    button_pin,
    GPIO.FALLING,
    callback = button_pressed,
    bouncetime = 300
)
```

```
Taster pritisnut - LED stanje True
```

```python
    # menjanje napona diode na logicku 1 ili 0 na osnovu led_state
    GPIO.output(led_diode, led_state) # 0/1
    # stampanje led_state u shell-u
    print(f"Taster pritisnut - LED stanje {led_state}")

GPIO.add_event_detect(
    button_pin,
    GPIO.FALLING,
    callback = button_pressed,
    bouncetime = 300
)

try:
    print("Program aktivan -> pritisni taster!")
    while True:
        sleep(1)
# 1. Nacin -> hvatanje greske preko CTRL+C
#except KeyboardInterrupt:
#    GPIO.cleanup()
# 2. Nacin -> finally blok koji se izvrsava bilo da je uhvacen izuzetak ili ne
finally:
    GPIO.cleanup()
```

```
Taster pritisnut - LED stanje True
Taster pritisnut - LED stanje False
Taster pritisnut - LED stanje True
Taster pritisnut - LED stanje False
Taster pritisnut - LED stanje True
Taster pritisnut - LED stanje False
```

Početak programa je isti kao sa prošlog primera, s tim što imamo globalnu promenljivu led_state koja je na početku postavljena kao False I ona služi za čuvanje logičkog stanja led diode. Definišemo funkciju button_pressed koja menja stanje globalne promenljive led_state odnosno invertuje vrednost I postavlja diodu na tu logičku vrednost koju ta promenljiva ima. Kako bi izvršili menjanje vrednosti globalne promenljive, potrebno je u funkciji koristiti ključnu reč global koja naznačava da se koristi globalna promenljiva I time izbegavamo kreiranje dodatne lokalne promenljive koja važi samo unutar funkcije, već menjamo vrednost direktno globalne promenljive. Na kraju funkcije u shell-u ispisujemo stanje led diode.

Nakon toga imamo događaj **add_event_detect()** u kojoj postavljamo o kom pinu se radi, da li se menja sa 0 na 1 ili sa 1 na 0, callback odnosno funkciju koju će da poziva I na kraju vreme odziva. U zadnjem delu programa vršimo delay svake jedne sekunde.